

Сборка кодинг-агента

День-0 база или жди сигнала — шесть развилок одного сквозного кейса

04

«Что в конфигурации агента стоит закладывать сразу, на день 0, как базовую практику — а что нельзя знать заранее, потому что оно специфично для вашего проекта?»

ПУСТАЯ ДИРЕКТОРИЯ → СЕМЬ ДОБАВЛЕНИЙ, ПОКА БЕЗ ИМЁН



РАЗДЕЛ 0 · ЗАДАЧА ОТ ЗАКАЗЧИКА

Заказчик формулирует задачу по-человечески, без единого слова о структуре, критериях готовности или границах действий — и с этим текстом вы открываете чат с агентом

«Нужен лендинг с формой заявки на демо-урок: одна статическая страница, поля 'имя' и 'email', кнопка отправки. Своего бэкенда нет — присылайте заявку на наш сервис приёма форм.»

«Можем ли мы сразу дать эту задачу агенту — или сначала нужно зафиксировать требования?»

открытая дискуссия, не голосование

Требования — функциональные и нефункциональные — раскладываются и записываются в `spec.md` до того, как агент увидит хоть строку задачи

```
# spec.md – signup-landing
```

Функциональные требования

- Статическая страница-лендинг, форма заявки на демо-урок
- Поля: имя (обязательное), email (обязательное)
- Кнопка отправки; без бэкенда – заявка уходит на внешний сервис

Нефункциональные требования

- Критерий приёмки: форма реально доставляет заявку – проверено на проде, а не только «тест зелёный»
- Граница действий: не деплоить на прод без явного запроса

«README.md и другие базовые артефакты создаются сейчас же, отдельно от `spec.md`. Спека — про требования к продукту. README — про устройство репозитория. Именно поэтому требования из `spec.md` потом не дублируются в файле инструкций агента.»

РАЗДЕЛ 0 · СКВОЗНОЙ КЕЙС

Репозиторий буквально пуст, а агент не задал ни одного уточняющего вопроса — и то, чего `спес.md` не покрывает, структуру, решил за вас молча

```
$ ls -la signup-landing/
total 8
drwxr-xr-x 2 harness harness 4096 Sep 20 12:45 .
drwxr-xr-x 3 harness harness 4096 Sep 20 12:45 ..
```

реальный вывод, не макет

«собери лендинг с формой заявки на демо-урок: статическая HTML-страница, сборка Vite (`npm run build` → `dist/`), поля формы — имя и email, тест на Playwright, который проверяет, что форма не отправляется с пустыми обязательными полями (`tests/form.spec.ts`)» — по содержанию совпадает со `спес.md`

«Уточняющий вопрос не задавался. Все параметры задачи... были однозначно выводимы из текста задачи, поэтому работа была начата сразу, без уточнений.» Но о структуре — как назвать файлы, куда их положить, как разложить проект — агент не спросил тоже: `спес.md` по определению это не описывает, это устройство кода, а не требование к продукту.

что создал

```
index.html
src/main.js
package.json
tests/form.spec.ts
# + спец.md, README.md (раздел 0) — агент их не
создавал
```

из финального ответа

```
npm install
npm run build      # → dist/
npm run test:e2e   # playwright test
```

Настраивая нового агента в новом репозитории, каждый ближе к одному из двух режимов

Прежде чем мы пойдём разбирать конкретные развилки — проверим на себе.

«Когда вы настраиваете нового агента в новом репозитории — какой из двух режимов вам обычно ближе?»

А · «Закладываю базу сразу»

гейт готовности («что считается сделано»)

трекинг задач и решений

В · «Жду, пока не заболит»

обзор репозитория

вложенные файлы конвенций

сложная, многоуровневая память

разбор — на следующем слайде

И то и другое — не ошибка, а привычка, которая права ровно наполовину: специфичное для проекта ждёт сигнала, универсальная гигиена заводится сразу

СПЕЦИФИЧНОЕ ДЛЯ ПРОЕКТА — ЖДАТЬ СИГНАЛА

обзор репозитория

вложенные файлы конвенций

сложная, многоуровневая память

УНИВЕРСАЛЬНАЯ ГИГИЕНА — ДЕНЬ 0

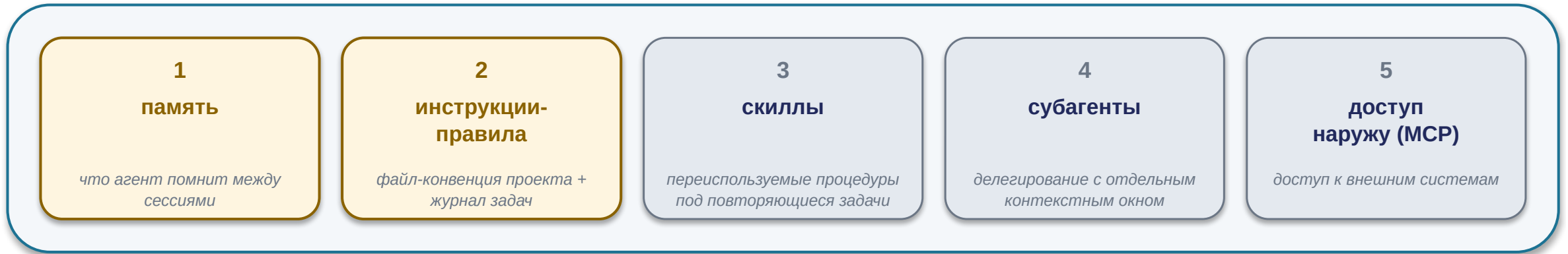
гейт готовности

трекинг задач и решений

«Это не то же самое, что ‘начинайте с малого всегда’. Лекция 3 говорила про тонкого агента — про отказ от структурной сложности. Это по-прежнему верно и по-прежнему ждёт сигнала. Но универсальная гигиена — другая категория, и здесь ‘жди боли’ — плохой совет. Сегодня — шесть развилок, ровно поровну.»

РАЗДЕЛ 0 · ОПОРНАЯ КАРТА

Из пяти слотов экипировки агента сегодня подробно разбираются два — инструкции-правила и память; остальные три и оба практических слоя — дальше, Семинаром 5



Сегодня подробно — два. Дальше, Семинар 5: скиллы, субагенты, доступ наружу + хук, процесс.

«Оставайтесь на самой нижней ступени, которая закрывает требование задачи. Поднимайтесь на следующую только тогда, когда можете назвать требование, которое текущая ступень не закрывает. Каждый подъём оплачивается новой стоимостью, новыми режимами отказа и новой поверхностью атаки.»

1

Раздел 1 · файл инструкций

- Кейс 1.1 · роль агента и репозиторий
- Кейс 1.2 · базовые процессы: гейты и цикл улучшения
- Кейс 1.3 · вложенные файлы по подпапкам

ОДИН ФАЙЛ, ТРИ РАЗВИЛКИ



Кейс 1.1 · роль агента и репозиторий

Что писать в пустой файл, пока не случилось ни одной реальной сессии

СПЕЦИФИЧНО ДЛЯ ПРОЕКТА · ЖДЁМ СИГНАЛА



КЕЙС 1.1 · РОЛЬ И РЕПОЗИТОРИЙ

Первый инстинкт — открыть пустой файл и начать с описания структуры репозитория; привычка сильная, но это ровно та фраза «так делают в хороших проектах», которую занятие проверяет впервые

дерево на входе в кейс

```
index.html
package.json
src/main.js
tests/form.spec.ts
# + spec.md, README.md (раздел 0) — здесь не
показаны
```

Разработчик открывает пустой CLAUDE.md и по привычке тянется первым делом написать «Repository overview» — из чего собран сайт, где лежит форма, какая структура папок.

«В одном из прошлых проектов раздел «Overview» пережил рефакторинг и два месяца спустя продолжал описывать структуру, которой больше не существовало — агент один раз отредактировал не тот файл, поверив описанию, а не факту.»

Перед пустым CLAUDE.md выбор из пяти вариантов — и «пока ничего» среди них не менее весомый вариант, чем остальные четыре

«Перед вами пустой CLAUDE.md для signup-landing. Выберите, что из перечисленного вы включите в файл прямо сейчас, до первой реальной сессии агента — из карточек ниже.»

подробное описание
структуры репозитория

список технологий
и зависимостей

инструкцию, где лежит
форма и как её найти

неочевидную
конвенцию,
которую агент не
угадает сам

пока ничего — до
первого реального
сигнала

разбор — на следующем слайде

Repository overview измеримо не помогает и стоит контекста — а структурированное описание должно жить в README.md, не дублироваться в CLAUDE.md

Источник	Тезис	Сила
Gloaguen et al., arXiv:2602.11988	«are not helpful» — шагов до файла не падает; cost +20–23% без прироста success	Сильное: контролируемый эксперимент
Та же работа, абляция	Убрали README — LLM-overview стал полезен (+2,7%)	Подтверждает механизм
Lulla et al., arXiv:2601.20404	AGENTS.md про недискаверабельные конвенции: −28,6% runtime (124 PR)	Умеренно «за» — не общее описание
Shepard & Albrecht, arXiv:2606.20512	Статичное guidance (28,3%) хуже верифицированного (33,0%, p<0,001)	Косвенное

Вариант	Где работает	Почему рано — что вместо
«Описание структуры»	Почти нигде	Агент выведет сам за секунды; устареет
«Список технологий»	Версии критичны, не видны	Обычно тоже выводимо
«Инструкция, где форма»	Почти нигде на малом репо	4 файла — весь экран
«Неочевидная конвенция»	Когда реально есть	Такой конвенции ещё нет
«Пока ничего — до сигнала»	Здесь и сейчас	ЦЕЛЕВОЙ ОТВЕТ

README, не AGENTS.md: структурированное описание должно существовать — но оно живёт в README.md (заведён в разделе 0), а не дублируется в CLAUDE.md. Именно поэтому «Repository overview» здесь не нужен — не потому что описание вообще не нужно, а потому что оно уже есть.

КЕЙС 1.1 · РЕШЕНИЕ

CLAUDE.md остаётся пустым для описания структуры репозитория — не из лени, а потому что нет сигнала; часть его содержимого заводится сразу, на дне 0, как дешёвая гигиена — это кейс 1.2

дерево репозитория — без изменений

```
index.html
package.json
src/main.js
tests/form.spec.ts
# CLAUDE.md — не создан
# + spec.md, README.md (раздел 0) — не в этом git-дереве
```

«Файл пуст не потому, что мы ленивы, а потому что для описания структуры репозитория нет сигнала. Но «пусто» не значит «навсегда»: часть содержимого CLAUDE.md мы заведём сразу, ещё на дне 0, как дешёвую гигиену — это следующий кейс.»

Кейс 1.2 · базовые процессы

Гейт-фраза и цикл «план → код → улучшение» — заведены с первого дня, и всё равно подводят

УНИВЕРСАЛЬНАЯ ГИГИЕНА · ДЕНЬ 0



Гейт готовности заведён с первого дня, вместе со `спес.md` — и всё равно «готово» расходится с реальностью дважды подряд

Гейт готовности заводится не когда прижмёт, а сразу, на день 0, вместе со `спес.md`: «Всегда прогоняй тесты и вручную проверяй результат перед тем как сказать 'готово'».

Две недели спустя: заказчик — «заявка не пришла». Агент читает код, правит обработчик, прогоняет тест — зелёный — «Готово». Тест проверяет только пустые поля. Разработчик напоминает про гейт — агент проверяет руками, чинит. Неделю спустя, другой баг — то же самое: тест зелёный, руками не проверено.

«Гейт был. Заведён правильно, с первого дня. Поведение — нет.»

Гейт-фраза уже есть с самого начала и не сработала — выбор из пяти вариантов, что делать дальше

«У вас гейт-фраза в CLAUDE.md есть с самого начала, а поведение агента всё равно расходится с ней второй раз подряд — заявка снова не проверена руками. Выберите, что вы сделаете дальше — из карточек ниже — так, чтобы это реально сработало в следующей сессии.»

перепишу фразу
капсом / жёстче

продублирую фразу
вторым пунктом

заведу отдельный
файл-чеклист рядом

передам проверку
механизму вне текста

смирюсь — сработает
не всегда, это норма

разбор — на следующем слайде

КЕЙС 1.2 · ИССЛЕДОВАНИЕ И РАЗБОР

Текстовый гейт без тулинга не просто бесполезен, а вреден — регрессии выросли почти до 10%; правила в промпте — просьбы, правила в коде — законы

Источник	Тезис	Сила
SWE-Gate/SpecBench/CapCode (4 препринта)	Согласие 4+ независимых исследований: агенты обходят видимые гейты	Сильное — консенсус
TDAD, arXiv:2603.17973	«Сначала пиши тесты» без тулинга — регрессии до 9,94%, хуже отсутствия	Сильное и контринтуитивное
Agent Scaffolding Beats Model Upgrades	Архитектура харнесса даёт +20 п.п.; смена модели — ~1 п.п.	Сильное — механизм против модели

Источник	Тезис	Сила
Self-planning, 2303.06689	Планирование перед кодом: +25,4% pass@1	Умеренно-сильное «за»
LLMs Cannot Self-Correct, ICLR'24	Самокоррекция без сигнала ухудшает	Сильное «против» self-review
arXiv:2604.10508	Self-repair с реальной проверкой: +4,9...+30 п.п.	Сильное «за» — с проверкой

Вариант	Где работает	Почему рано — что вместо
«Капсом / жёстче»	Нигде системно	TDAD: регрессии выросли до 9,94%
«Продублирую вторым пунктом»	Иногда чуть повышает шанс прочитать	Presence paradox: лишняя строка стоит контекста
«Отдельный файл-чеклист»	Помогает человеку, слабо — агенту	Тот же класс решения — снова текст, снова просьба
«Передам механизму вне текста»	Системно работает: +20 п.п.	Верное направление — но не тема сегодня, Семинар 5
«Смирюсь, это нормально»	Приемлемо для некритичных предпочтений	Для правила, чьё нарушение стоит дорого — решение не решать

Правила в промпте — просьбы, правила в коде — законы. Текстовый гейт — просьба, и сегодня мы записываем именно просьбу. Цикл «улучшение»: «посмотри на свой код» — не делать; «прогони реальную проверку» — делать всегда.

Итоговый файл — тринадцать строк: критерий готово, гейт-предупреждение, честная оговорка предела; AGENTS.md — симлинк на тот же файл

CLAUDE.md

signup-landing: статический лендинг с формой заявки на демо-урок.
Готово = npm run build код 0, npx playwright test зелёный,
форма реально отправляет заявку на проде.

Safety / scope boundaries

- Никогда не запускать деплой на прод без явного запроса.

Build, test, verify

- To verify: открыть dist/index.html после сборки, вручную отправить форму с заполненными и с пустыми полями.

1

Файл грузится в начало каждой сессии целиком (официальный ориентир — до 200 строк, наш итог — 13)

2

@-импорты организуют файл, но не экономят контекст

3

AGENTS.md — симлинк на тот же файл (ln -s CLAUDE.md AGENTS.md). Острый край: cp -R без -P на macOS разыменовывает симлинк во вторую копию молча — копировать через cp -a / git clone / git archive

Кейс 1.3 · вложенные файлы

Месяц спустя репозиторий попрос — дробить файл или нет

СПЕЦИФИЧНО ДЛЯ ПРОЕКТА · ЖДЁМ СИГНАЛА



КЕЙС 1.3 · ВЛОЖЕННЫЕ ФАЙЛЫ

Разработчик выносит правила тестов в tests/CLAUDE.md и ждёт, что агент подхватит его сам — логично звучит, но стоит проверить, что там реально происходит с загрузкой

месяц спустя: tests/ подрос

tests/

form.spec.ts

wizard.spec.ts

page-objects/form.page.ts

своя конвенция именования + Page Object

«Вынесу эти правила в отдельный tests/CLAUDE.md — короче, релевантнее контексту, корневой файл не разбухнет.» Кладёт файл в подпапку, ожидая автоматической подхватки.

«Логично звучит. Давайте проверим, что там на самом деле происходит с загрузкой.»

Как гарантировать, что агент реально прочитает конвенции из tests/CLAUDE.md — выбор из пяти вариантов

«Выберите, как вы гарантируете, что агент реально прочитает конвенции из tests/CLAUDE.md, когда работает именно в этой папке — из карточек ниже.»

положу файл в tests/
и доверюсь
автозагрузке

явно попрошу агента
прочитать файл в
начале задачи

продублирую
содержимое
в корневом CLAUDE.md

проверю через /context,
что реально загружено

объединю всё в один
файл в корне, пока не
мешает

разбор — на следующем слайде

КЕЙС 1.3 · ИССЛЕДОВАНИЕ И РАЗБОР

Вложенный файл грузится on-demand, только по факту чтения; контролируемое исследование не нашло пользы от разбиения, а практика показывает регулярные отказы

Механизм: корневой CLAUDE.md грузится при старте сессии автоматически. Вложенные — только on-demand, по факту чтения. /context — единственный надёжный способ проверить.

Источник	Тезис	Сила
McMillan, 2605.10039 (1650 сессий)	Ни одна из 4 структурных переменных не дала эффекта	Сильное и контринтуитивное
Claude Code #2571 / #6972	Не подхватывается автоматически; документация вводит в заблуждение	Прямое наблюдение
Cursor / Codex CLI	Bag nested-функциональности / ненадёжная подгрузка	Прямое, другие инструменты
GitHub Copilot CLI #3051	Документированный структурный предел discovery	Самое жёсткое — архитектурный предел

Вариант	Где работает	Почему рано — что вместо
«Доверюсь автозагрузке»	Никогда гарантированно	Вложенные файлы грузятся on-demand. Задокументированные отказы — системная категория
«Явно попрошу прочитать»	Работает, требует ручной дисциплины	Надёжнее автозагрузки, но перекладывает ответственность на человека
«Продублирую в корневой»	Работает всегда — корень грузится при старте	Снова presence paradox: продублированная строка стоит контекста везде
«Проверю через /context»	Единственный способ узнать факт	Диагностика, не решение
«Объединю в один файл в корне»	Здесь и сейчас — репозиторий маленький	Целевой ответ. Исследование не нашло значимого эффекта ни у одной структурной переменной

Пока корневой файл помещается в разумный объём (до 50–70 строк), дробление не нужно никогда. Даже при росте — только после явной проверки через /context.

КЕЙС 1.3 · РЕШЕНИЕ

Репозиторий остаётся с одним корневым файлом; три кейса — три решения знания, известного заранее, но есть знание другого рода, которое появляется по ходу работы — это раздел второй, память

дерево репозитория — без изменений

AGENTS.md

CLAUDE.md

index.html

package.json

src/main.js

tests/form.spec.ts

tests/CLAUDE.md — не появляется

+ spec.md, README.md (раздел 0) — не в этом git-дереве

«Три кейса, один и тот же файл: мы решили, чего в него не писать заранее; написали то, что узнали на реальной боли, даже когда завели гейт заранее; и решили не дробить его, пока это не доказанная польза. Но есть знание другого рода, которое появляется по ходу самой работы. Это раздел второй — память.»

2

Раздел 2 · память

Кейс 2.1 · плоский файл, день 0

Между сессиями агент не хранит ни бита — если это не лежит на диске

- Кейс 2.1 · плоский файл
- Кейс 2.2 · структурированная вики-память
- Кейс 2.3 · операционная память

УНИВЕРСАЛЬНАЯ ГИГИЕНА · ДЕНЬ 0



КЕЙС 2.1 · ПЛОСКИЙ ФАЙЛ

Файл **DECISIONS.md** заведён с первого дня — и нужной записи в нём всё равно нет, потому что решение родилось в разговоре, а не за столом

Третья сессия за две недели. Заказчик прислал заявку с опечаткой в домене почты, форма её приняла. Агент предлагает подключить стороннюю библиотеку валидации форм — ту самую, которую уже обсуждали и отклонили: форма из двух полей, хватает нативных `required/pattern`.

«Файл **DECISIONS.md** уже есть — заведён с первого дня, вместе со `спес.md` и `README`. И тем не менее нужной записи в нём нет: решение родилось не за столом, а на бегу, в потоке рабочего разговора. Завести практику — не то же самое, что воспользоваться ею в момент, когда это имеет значение.»

Выберите, куда записать отклонённое решение — из карточек ниже — так, чтобы не объяснять его в третий раз

напоминание: отклонённая библиотека валидации всплыла снова — решение родилось в разговоре и нигде не осталось, даже при уже заведённом DECISIONS.md

«Вы второй раз за две недели объясняете агенту, почему в этой форме нет сторонней библиотеки валидации. Выберите, куда записать это решение — из карточек ниже — так, чтобы не объяснять его в третий раз, и обоснуйте, почему не в тот файл инструкций, который мы только что завели.»

дописать
в CLAUDE.md

сказать агенту
«запомни это»

записать в
DECISIONS.md — он уже
есть

поставить систему
памяти

ничего — сам
запомнит

разбор — на следующем слайде

КЕЙС 2.1 · ИССЛЕДОВАНИЕ

Presence paradox из кейса 1.1 применяется и здесь: то, что лежит в DECISIONS.md, стоит контекста в каждой сессии независимо от того, нужна ли эта запись сегодня

Кейс 1.1: Gloaguen et al. — сам факт присутствия текста в контексте создаёт стоимость (+20–23%), независимо от того, помогает ли он решить задачу. Мы назвали это presence paradox.

DECISIONS.md — не исключение. Каждая строка грузится в контекст в каждой следующей сессии, не только когда она реально нужна. Тридцать записей о разных темах — цена уже заметна, даже если задача касается только одной из них.

прямого измерения именно на журналах решений не проводилось — это перенос уже установленного механизма на новый объект, не отдельное новое исследование

если запись не бесплатна — точно ли туда идёт вообще всё, что можно туда положить?

Командное решение с обоснованием идёт в DECISIONS.md, личное предпочтение — в авто-память, выводимое из кода — никуда

вопрос для контекста: «Куда записать отклонённое решение — и почему не в файл инструкций?»

Вариант	Где это работает	Почему здесь ещё рано — и что вместо
«Допишу в CLAUDE.md»	Механически сработает: файл читается каждую сессию	Смешивает стабильные правила и растущий лог решений. На десятке решений файл выйдет за ориентир в 200 строк
«Ничего, агент сам запомнит»	Нигде: это не настройка, которую забыли включить	Модель не имеет состояния между вызовами по конструкции
«Скажу агенту ‘запомни это’»	Для личного предпочтения — идеально	Половина ответа. Уходит в авто-память: машинно-локальный слой без синхронизации и код-ревью
«Заведу DECISIONS.md»	Для командного решения с обоснованием — именно то, что нужно	Вторая половина ответа. Требуется писать «почему», а не «что»
«Поставлю систему памяти»	На десятках-сотнях записей, нескольких пользователях	Рано на порядок: на малых корпусах простой файл регулярно конкурентен со сложными системами

целевой ответ — оба механизма разом, с разделением ролей

Не всё, что узнал агент, вообще стоит записывать — третья корзина такая же законная, как первые две

АВТО-ПАМЯТЬ

Разработчик просит отвечать
коротко, без преамбул

DECISIONS.MD

Не подключаем стороннюю библиотеку
валидации — форма из двух полей

Своего бэкенда не делаем — заявка
уходит на внешний сервис приёма форм

НИКУДА

Обработчик формы лежит
в src/main.js

19 сентября тест упал из-за
таймаута, увеличили ожидание

собранная запись — реальный файл демо-репозитория

2026-09-20 — не подключаем библиотеку валидации форм

Форма — два поля (имя, email), хватает нативных required/pattern в
index.html. Сторонняя библиотека — лишняя зависимость ради этого объёма.

Новое — не файл (он с нами с первого дня), а первая содержательная запись в нём

DECISIONS.md — целиком

Decisions

Append-only log of why, in date order.

2026-09-20 — не подключаем библиотеку валидации форм

Форма — два поля (имя, email), хватает нативных required/pattern в index.html. Сторонняя библиотека — лишняя зависимость и лишние килобайты в сборке ради этого объёма.

дерево кейса 2.1

AGENTS.md

CLAUDE.md

DECISIONS.md

index.html

package.json

src/main.js

tests/form.spec.ts

+ spec.md, README.md (раздел 0) — не в этом git-дереве

Разница между слоями не в формате — оба markdown, — а в том, кто инициирует запись и кто её потом увидит

	Файл инструкций	Авто-память	DECISIONS.md
Кто автор	вы, заранее	агент, по ходу	вы, в момент решения
Где живёт	в репозитории, git	вне репозитория, на этой машине	в репозитории, git
Что грузится	целиком, каждую сессию	индекс: первые 200 строк / 25 КБ	ничего автоматически
Проходит код-ревью	да	нет	да
Переживает смену машины	да	нет	да
Что туда идёт	стабильные правила и границы	предпочтения, подтверждённые подходы	решения и их «почему»

индекс памяти — только индекс · агент сам пропускает выводимое из кода · сжатие контекста переживают только файлы на диске

Память по умолчанию читается как доверенный контекст, а не как данные — и включение памяти не обязательно улучшает результат



Отравление памяти — SpAIware (сентябрь 2024)

Внешний текст с вложенной инструкцией → долговременная память → активация в последующих сессиях, эксфильтрация переписки. Вектор закрыт в версии 1.2024.247; структурный принцип остался.

≈48% всех последующих обращений к памяти захвачено (MemoryGraft, 2025; до отравления — 0%). В смежном эксперименте 5 подложенных документов управляют выводом в >90% случаев.

Замер	Значение	База сравнения
Популярная система памяти на бенчмарке долговременной памяти	46,0% с памятью против 57,6% без	та же система, память выключена
Шесть систем памяти против «весь текст сессии в один файл»	проигрыш плоскому файлу на 25–67 пунктов	тот же плоский файл
Дискреционность записи	17% (4 задачи из 24) — модель отказалась записать нужный факт	те же 24 задачи

Плоского файла достаточно, пока история решений влезает в контекст целиком и связей между записями ещё нет

- Задача одноразовая, второй сессии по этому контексту не будет.
- Весь нужный контекст помещается в одно контекстное окно без потерь.
- Правила проекта стабильны и укладываются в файл инструкций короче 200 строк.
- Никто не повторяет вручную одну и ту же правку из сессии в сессию.
- Нет нескольких участников, которым нужно делиться контекстом асинхронно.
- Факт выводим из кода или истории изменений.

Плоский DECISIONS.md перестаёт справляться, когда записей набралось настолько много, что найти нужную линейным чтением дольше, чем переобъяснить решение заново, — и между записями появились связи, для которых в append-only логе физически некуда положить.

Кейс 2.2 · структурированная вики-память

Тот же DECISIONS.md, но разросшийся — плоский лог больше не хранит связей

СПЕЦИФИЧНО ДЛЯ ПРОЕКТА · ЖДЁМ СИГНАЛА



Файл технически на месте, но пользоваться им дольше, чем не пользоваться — линейный лог не хранит связей между записями

Прошло четыре месяца. Новый разработчик в команде просит агента добавить капчу на форму: появился спам. Агент листает DECISIONS.md целиком (тридцать восемь записей) и предлагает решение, которое противоречит более раннему решению №14: «не добавляем сторонние виджеты в форму, см. запись №3». Агент нашёл запись №3, но не нашёл запись №31 — она отменяет часть решения №3 конкретно для случая антиспама, дописана двумя месяцами позже, в другом конце файла, без единой отсылки назад.

Разработчик открывает файл сам и тратит на линейное чтение тридцати восьми записей больше времени, чем ушло бы на то, чтобы заново передумать вопрос с нуля.

Пример иллюстративный — реального разросшегося DECISIONS.md на 38 записей не захвачено.

Разросшийся журнал решений и потерянная связь между двумя записями — повод выбрать, как его реорганизовать, а не заводить заново

«DECISIONS.md разросся, и в нём уже есть решение, которое отменяет часть более раннего решения, — без единой ссылки между ними. Выберите, как реорганизовать этот файл — из карточек ниже — так, чтобы агент быстро находил нужное решение и видел связи между решениями, а не листал линейный лог с начала. Обоснуйте выбор.»

оставить как есть —
просто грепать

разбить на
тематические
файлы со ссылками

поставить графовую/
векторную систему
памяти

завести один общий
SUMMARY.md,
переписываемый
целиком

агент сам найдёт
через поиск по
репозиторию

разбор — на следующем слайде

Плоская память деградирует со временем, а структура может окупаться именно для агента там, где не окупается для человека — но «умнее» не значит «лучше»

почти все источники ниже — свежие непроверенные препринты 2026 года; цифры — «заявлено в препринте», а не установленный факт

MENTIER (arXiv:2605.03675): плоская память деградирует на горизонте порядка 72 часов работы — минус 14 п.п. успешности вызовов инструментов (насыщение контекста, temporal decay, semantic drift). Логика «raw sources → wiki → schema»: у человека вики умирает — поддержка дороже ценности; у агента стоимость правки полутора десятков файлов близка к нулю.

Система	LoCoMo (долговременная память)
Простое файловое хранилище (grep/чтение файлов)	74,0%
Mem0 (graph-based)	68,5%

Летта-контрпример: простое хранилище обошло графовую систему памяти — модели тренированы на файловых операциях как на самом привычном инструменте. Держите эту цифру в голове — она вернётся в разборе.

Разбить на тематические файлы со сквозными ссылками — целевой ответ, но не бесплатный: структура сама создаёт новую нагрузку

вопрос для контекста: «Как реорганизовать разросшийся журнал решений?»

Вариант	Где это работает	Почему здесь ещё рано — и что вместо
«Оставить как есть, просто грепать»	Пока записей десятки и они укладываются в контекст целиком	Деградация плоской памяти со временем задокументирована (~72 ч, −14 п.п.); наш файл уже перерос эту границу
«Разбить на тематические файлы со сквозными ссылками»	Именно этот случай	Целевой ответ, но не бесплатный — структура сама создаёт новую нагрузку (staleness, дублирование, поддержка)
«Поставлю графовую/векторную систему памяти»	На больших многопользовательских корпусах с семантическим поиском	Контрпример прямо про наш масштаб: файловое хранилище обошло graph-based систему на LoCoMo
«Один общий SUMMARY.md, переписываемый целиком»	Для совсем маленького проекта	Переписывание целиком убивает append-only гарантию
«Агент сам найдёт через поиск по репозиторию»	Для фактов, выводимых из кода	Решения — не факты кода. «Сам найдёт» — то же заблуждение, что «сам запомнит»

целевой ответ — разбить на тематические файлы со сквозными ссылками, со списком реальных издержек, а не как безусловный апгрейд

Запись явно ссылается на связанную запись — то, чего в плоском append-only логге не было в принципе

`memory/decisions/`

<code>index.md</code>	← оглавление тем + ссылки
<code>validation.md</code>	← решения по валидации формы
<code>backend.md</code>	← решения о том, чего мы НЕ делаем
<code>deploy.md</code>	

`validation.md` — пример записи

2026-09-20 — не подключаем библиотеку валидации форм

Форма — два поля, хватает нативных `required/pattern`.

См. также: 2026-11-30 (антиспам) — это решение НЕ

распространяется на защиту от спама, другой механизм.

2026-11-30 — антиспам через `honeypot`-поле, без капчи

Отменяет часть решения от 2026-09-20 для случая антиспама.

иллюстративный пример, не захваченный вживую — ждёт реальной сессии

КЕЙС 2.2 · ПРОВАЛ И ИЗДЕРЖКИ

Структура — не гарантированный выигрыш и не бесплатный апгрейд: простое файловое хранилище обошло графовую систему памяти, а сама структура добавляет свою собственную нагрузку

почти все источники ниже — свежие непроверенные препринты 2026 года; цифры — «заявлено в препринте», а не установленный факт

Система	LoCoMo (долговременная память)
Простое файловое хранилище (grep/чтение файлов)	74,0%
Mem0 (graph-based)	68,5%

объяснение авторов: модели тренированы на файловых операциях как на самом привычном инструменте

ЧЕСТНО НАЗВАННЫЕ НЕДОСТАТКИ ВИКИ-ПАМЯТИ

Staleness — сводка в оглавлении расходится с содержимым тематического файла. · Duplication — устаревшие версии одного и того же факта накапливаются вместо замены. · Отсутствие каскадного удаления — удалили факт в одном файле, производные записи в других остались жить как верные. · Maintenance burden — кто-то должен реально ходить и поддерживать структуру актуальной.

КЕЙС 2.2 · КРИТЕРИЙ И МОСТ

Единиц-десятков тематических файлов, которые правят один-два человека, достаточно — полноценная система памяти требует замеров на своём масштабе, не общего совета

- Число тематических файлов — единицы-десятки, не сотни.
- Поиск нужной записи — по имени темы или простому grep, а не по смысловой близости.
- Участников, одновременно правящих память, — один-два.
- Никто не просил «найди решения, похожие по смыслу на...».

Смена оси: всё в этом и предыдущем кейсе — про решения, которые живут навсегда. Что происходит с памятью одной задачи, пока она ещё не закончена?

Кейс 2.3 · операционная память

Смена оси: не решения навсегда, а ход одной задачи между её сессиями

УНИВЕРСАЛЬНАЯ ГИГИЕНА · ДЕНЬ 0



КЕЙС 2.3 · ЗАДАЧНАЯ ПАМЯТЬ

Файл задачи существовал с самого начала, план в нём выглядит актуальным — и всё равно вторая сессия предлагает переписать то, что уже негласно решили не трогать

Команда завела практику «файл на задачу» с первого дня работы над signup-landing — как и DECISIONS.md, это дешёвая гигиена, которую заводят сразу. Разработчик ставит агенту задачу на несколько сессий: разбить форму на пошаговый визард, сохранив всю текущую валидацию. По заведённой практике агент перед началом сам создаёт файл задачи с планом. Первая сессия закрывает первые шаги. По ходу — не по плану, а в разговоре — разработчик и агент договариваются не трогать email-валидацию. Договорённость мелкая, кажется не стоящей того, чтобы прерваться и записать её прямо в моменте. Сессия обрывается на середине следующего шага: контекст сжимается. Вторая сессия — файл на месте, выглядит актуальным и достоверным, но договорённость про email-валидацию в лог так и не попала. Агент предлагает переписать email-валидацию заново.

«Файл был. Практика была. Не хватило одного: вписать мелкую договорённость в лог в момент, когда она прозвучала.»

Файл задачи существовал с самого начала и всё равно ввёл в заблуждение — выбор из пяти вариантов

«Файл задачи существовал с самого начала, план в нём выглядит актуальным — первые шаги отмечены сделанными. И всё равно вторая сессия предлагает переписать то, что уже негласно решили не трогать. Выберите, что нужно сделать иначе — из карточек ниже — так, чтобы файл задачи не вводил в заблуждение, а реально отражал ход работы. Обоснуйте выбор.»

надеяться на
контекстное окно

записать план
в DECISIONS.md

дисциплинированно
дописывать
лог хода сразу

завести папку на задачу
с файлом под каждый
шаг

попросить агента
«продолжи с того
места»

разбор — на следующем слайде

КЕЙС 2.3 · ИССЛЕДОВАНИЕ

От Pokémon до recitation — практика структурированных заметок о ходе задачи известна и работает; и она объясняет, почему наш файл всё равно подвёл

Anthropic, инженерный блог: «structured note-taking» — агент играет в Pokémon тысячи шагов подряд, ведя простые структурированные заметки, переживая сбросы контекста без потери прогресса. Manus, «recitation»: агент непрерывно переписывает todo.md в конец контекста — снижает уход от цели. metasphere-agents: файл на задачу в .tasks/active/, архивируется в .tasks/done/ по завершении.

«Есть измерение на большом числе траекторий: план в среднем помогает. Но дословная цитата авторов — ‘плохой план вреднее отсутствия плана вообще’. Полную картину с цифрами покажу на слайде провала.»

Дисциплина синхронной записи в уже существующий файл на задачу — целевой ответ; файл задачи это не то же самое, что журнал решений

вопрос для контекста: «Файл задачи ввёл в заблуждение — что сделать иначе?»

Вариант	Где это работает	Почему рано — и что вместо
«Не сжимать, не закрывать сессию»	Для задачи, целиком укладываемой в одну короткую сессию	Контекстное окно конечно; на многошаговой задаче рано или поздно случится сжатие или разрыв сессии — это отсутствие стратегии
«Записать план в DECISIONS.md»	Никогда — не тот слой памяти	DECISIONS.md — про решения, которые остаются в силе навсегда; статус одной задачи туда не относится
«Дисциплинированно дописывать лог хода сразу»	Ровно наш случай	Целевой ответ для этого масштаба
«Папка на задачу с файлом под каждый шаг»	Параллельные подзадачи, несколько субагентов, разросшийся лог	Оверинжиниринг для одной линейной задачи здесь и сейчас
«Попросить ‘продолжи с того места’»	Внутри одной непрерывной сессии, до сжатия	После сжатия или в новой сессии буквально нечего продолжать

целевой ответ — дисциплина синхронной записи в уже существующий файл, с явной оговоркой об упрощении до файла

Один и тот же файл, одна и та же практика — разница ровно в том, записали вовремя или нет

```
.tasks/active/signup-wizard.md
```

```
# Задача: разбить форму на пошаговый визард
```

```
## План
```

1. ☒ Разметка трёх шагов (имя / email / подтверждение)
2. ☒ Перенос текущей валидации без изменений
3. ☐ Состояние текущего шага (JS)
4. ☐ Кнопки «назад»/«далее»
5. ☐ Тесты на переключение шагов
6. ☐ Ручная проверка полного прохождения

```
## Лог хода
```

- 2026-09-20: шаги 1–2 закрыты. Договорились НЕ трогать email-валидацию — обсуждалась в DECISIONS.md (2026-09-20), тут перенос.
- 2026-09-22: начат шаг 3, не закончен — контекст сжат в середине.

```
## Результаты (заполняется по завершении)
```

```
—
```

иллюстративный пример, не захваченный вживую — ждёт реальной многошаговой сессии владельца · упрощение до файла — масштабируется до папки, подробнее за пределами этого занятия

Плохой план хуже отсутствия плана вообще — и это подтверждено количественно, а не только интуицией

Источник	Результат
Evaluating Plan Compliance, 21 120 траекторий, SWE-bench Verified/Pro	«bad plans hurt performance more than no plan at all»; комплаенс к плану падает ~на 13 п.п. на сложном бенчмарке
Fresh Memory, Stale Plans, 30 контролируемых живых сценариев	без проверки актуальности плана исполнитель действовал по устаревшему плану во всех 30 случаях (100%)



Кейс-стади «Index Sickness» (n=1, честная оговорка)

Реальный проект, 391 сессия за месяц. Решение — физическое разделение baseline-инструкций и append-only лога сессии. Результат: объём AI-инструкций сократился ~на 75%, рецидивов не было за следующие ~150 сессий. n=1 action research одного автора, не контролируемый эксперимент.

практика «файл/папка на задачу» пока не мейнстрим — 2853 реальных репозитория, доминируют статические context-файлы

КЕЙС 2.3 · КРИТЕРИЙ

Одного файла на задачу достаточно, пока задача линейная и её ведёт один агент и один разработчик — иначе оправдан переход к папке

- Задача линейная: шаги идут по порядку, не параллельно.
- Работает один агент (без субагентов/параллельных веток) и один разработчик.
- Лог хода помещается в разумный файл (сотни строк, не тысячи) без потери читаемости.
- Результатов и артефактов на выходе немного — не нужен отдельный файл на каждый.

Если хотя бы половина пунктов неверна — переход к папке на задачу оправдан.

честная оговорка: паттерн «файл/папка на задачу» пока не мейнстрим (2853 реальных репозитория, доминируют статические context-файлы) — практику продвигают отдельные лидеры, это не общепринятая норма

РАЗДЕЛ 3 · СИНТЕЗ

Шесть развилок сегодняшнего занятия делились на два типа: специфичное для проекта (ждать сигнала) и универсальная гигиена (заводить сразу, на день 0)

Кейс	Развилка	Что мы записали	Когда
1.1	Роль агента и общее описание репозитория	Ничего — до первого реального сигнала	по сигналу
1.2	Базовые процессы: гейт «готово» + цикл улучшения	Гейт-фраза текстом в CLAUDE.md	сразу
1.3	Вложенные файлы инструкций по подпапкам	Не дробим — один файл в корне	по сигналу
2.1	Плоский файл памяти (DECISIONS.md)	Решение записано в файл + авто-память	сразу
2.2	Структурированная вики-память	Тематические файлы со сквозными ссылками	по сигналу
2.3	Операционная задачная память	Один файл на задачу: план + лог + результаты	сразу

Три развилки ждали реального сигнала. Три — не нуждались в сигнале вообще, их стоило завести сразу.

Сегодня закрыты подробно два слота харнесса из пяти; оставшиеся три плюс оба механических слоя, которые превращают просьбу в барьер, — материал Семинара 5

1

память

что агент помнит между сессиями

2

**инструкции-
правила**

*файл-конвенция проекта +
журнал задач*

3

скиллы

*переиспользуемые процедуры
под повторяющиеся задачи*

4

субагенты

*делегирование с отдельным
контекстным окном*

5

**доступ
наружу (MCP)**

доступ к внешним системам

Семинар 5: скиллы · субагенты · доступ наружу (MCP) + хук · процесс

РАЗДЕЛ 3 · ПЕРЕНОС

Перенесите шесть сегодняшних решений на свой репозиторий — и проверьте, какое из них реально день-0-база, а какое вы отложили без причины или завели раньше времени без сигнала

1.1 → 1.2 → 1.3 → 2.1 → 2.2 → 2.3

«По каждой развилке спросите: это действительно день-0-база — то, что стоило завести сразу, независимо от конкретики проекта? Или вы завели это раньше времени, без реального сигнала, просто по привычке? А может, наоборот — что-то из универсальной гигиены вы всё ещё откладываете без всякой причины?»

«Не вслух. Не на бумаге. Себе.»